Relatório de Continuidade — Checklist de Pendências BrismarPay

Data: 26/08/2026
Escopo: melhorias independentes da Getnet, mantendo `PAYMENT_PROVIDER=mock`
Base: projeto BrismarPay e checklist de pendências fornecidos pelo usuário

Resumo

Foi realizada uma nova rodada de implementação sem depender das informações da Getnet. O objetivo foi avançar as pendências internas que possuem contrato suficiente no próprio projeto, sem inventar endpoints, payloads, credenciais ou regras comerciais da adquirente.

Área	Resultado
API Keys	Implementadas e módulo habilitado
Segurança das chaves	Hash SHA-256, segredo exibido uma única vez e revogação lógica
Guard de API Key	Implementado para `X-Api-Key` e `Authorization: Bearer`
CI	Pipeline GitHub Actions criado com Node, Prisma, PostgreSQL, Redis, build, unitários e E2E
Provider padrão	`mock`, preservado
Build	Aprovado
Testes unitários	9 suítes e 65 testes aprovados
E2E local	13 cenários ignorados por ausência de PostgreSQL/Redis locais

Implementações realizadas nesta continuidade

API Keys e API pública

O `ApiKeysService` agora gera chaves sandbox e produção com aleatoriedade criptográfica, persiste apenas o hash SHA-256 e retorna o segredo em claro somente na resposta de geração ou rotação. A listagem retorna exclusivamente metadados, a revogação preserva o registro e a rotação revoga a chave anterior e cria a nova dentro de uma transação.

O `ApiKeyAuthGuard` valida `X-Api-Key` ou Bearer, rejeita chaves inválidas e revogadas, atualiza `lastUsedAt` de forma assíncrona e anexa o contexto da empresa autenticada à requisição. O módulo foi registrado no bootstrap do backend. As rotas administrativas de gerenciamento continuam protegidas por JWT e RBAC.

CI e ambiente limpo

Foi criado `.github/workflows/ci.yml`. O pipeline instala dependências com `npm ci`, gera o cliente Prisma, aplica o schema em PostgreSQL descartável, executa o build, a suíte unitária com cobertura e os testes E2E com Redis e PostgreSQL descartáveis. O workflow define `PAYMENT_PROVIDER=mock` e um segredo JWT exclusivo do CI, sem inserir credenciais reais.

Documentação

`docs/FASE_7_API_TODO.md` foi atualizado para registrar o que já está implementado e delimitar o que ainda depende da definição das rotas públicas, escopos por endpoint, rate limiting específico, logs detalhados e webhooks de saída.

Validações

O build TypeScript foi concluído com sucesso. A suíte unitária completa foi concluída com 65 testes aprovados em 9 suítes. A suíte E2E está configurada e foi detectada localmente, mas permanece ignorada porque o ambiente desta execução não possuía `TEST_DATABASE_URL` e `TEST_REDIS_URL`; no CI criado, esses serviços são disponibilizados por containers descartáveis.

Foi feita varredura por padrões usuais de chaves privadas e tokens conhecidos nos arquivos do projeto, sem resultados relevantes. O ZIP final foi testado com `unzip -t` e não inclui `node_modules`, `dist`, `.next` ou cobertura gerada.

O que ainda não pode ser concluído sem a Getnet

Continuam bloqueados o contrato real do produto Pix, URLs oficiais por ambiente, schema definitivo, campos de idempotência, status e substatus, autenticação oficial de webhooks, payloads de homologação, regras de expiração, conciliação financeira e aceite formal dos cenários. A implementação preparada no adaptador Getnet depende dessas informações e não deve ser ativada antes da homologação.

Também permanecem como próximos trabalhos internos: dispatcher de webhooks de saída com retry e assinatura, métricas persistentes e alertas, conciliação persistida com divergências,

notificações, regras comerciais de taxas, API pública de negócio com escopos por rota, documentação OpenAPI e configuração de proteção de branch/review no GitHub.

> A ausência das informações da Getnet não impede o avanço interno, mas impede declarar a integração real homologada ou pronta para produção.

Arquivos principais alterados nesta continuidade

Arquivo	Alteração
`apps/api/src/modules/api-keys/services/api-keys.service.ts`	Geração, listagem, revogação, rotação e autenticação de chaves
`apps/api/src/modules/api-keys/guards/api-key-auth.guard.ts`	Guard de autenticação por API Key
`apps/api/src/app.module.ts`	Registro do módulo de API Keys
`.github/workflows/ci.yml`	Pipeline automatizado com PostgreSQL/Redis e E2E
`docs/FASE_7_API_TODO.md`	Documentação atualizada da Fase 7

Próximo passo recomendado

Executar o pipeline em um repositório GitHub real, revisar os cenários E2E com a equipe e implementar os módulos internos restantes. Quando a Getnet fornecer o pacote técnico, preencher as variáveis oficiais, criar fixtures anonimizadas, executar os testes de contrato e somente então iniciar a homologação.